



Vandana Yarala

Posted on 28 Jun

Edit Manage Stats

Building TrialBridge AI with Vercel and AWS Aurora PostgreSQL

#aws #vercel #postgres #h0hackathon

[TrialBridge AI](#) is a B2B web application that helps clinical research teams match patients to clinical trials and coordinate the human follow-up work that comes after. This post walks through why we built it, what it does, the architecture, and the engineering decisions behind it including the part that took the most iteration: connecting Vercel to AWS Aurora PostgreSQL with **passwordless IAM authentication** over OIDC.

Disclaimer: TrialBridge AI is a research-workflow tool. It does not diagnose, treat, prescribe, or make final medical decisions, and every patient record in the demo is synthetic. That constraint shaped the whole design, as you'll see in the safety section.

1. Why we built [TrialBridge AI](#)

Clinical trial recruitment is slow and expensive. Research coordinators spend enormous amounts of time reading trial eligibility criteria, cross-referencing patient charts, and figuring out who *might* qualify — work that is repetitive, error-prone, and hard to audit. Many promising trials fail to enroll enough participants, and many patients never learn about trials they could have joined.

We didn't want to build "an AI that decides who is eligible." That's the wrong tool for a regulated, high-stakes domain. Instead, we wanted to build the layer underneath that decision: a system that **organizes the information, applies the trial's own published criteria transparently, explains its reasoning, and hands a clean, prioritized case to a human coordinator** for review and referral.

In short: keep the human in the loop, but give them a much faster, more structured starting point.

2. What the application does

[TrialBridge AI](#) is organized around a few core workflows, each backed by its own page in the app:

- **Trials** — a catalog of clinical trials with their structured inclusion/exclusion criteria (condition, phase, location, sponsor, and a list of weighted rules).
- **Patients** — synthetic patient profiles with demographics, a condition, free-text summary, structured attributes, and lab results.
- **Matching** — for any patient, the app evaluates them against a trial's criteria and produces a status (`likely_eligible`, `possibly_eligible`, or `not_eligible`), a numeric score, and a per-criterion explanation.
- **Coordinator tasks** — matches that need human attention become prioritized tasks with status tracking (`new` → `needs_review` → `contact_pending` → `referred_to_pi` → `closed`).
- **Audit log** — every meaningful action is recorded as an audit event, so the workflow is reviewable after the fact.

The key idea is that the output is never "this patient is eligible." It's "here is how this patient lines up against each published criterion, here's what's missing, and here's what a coordinator should look at next."

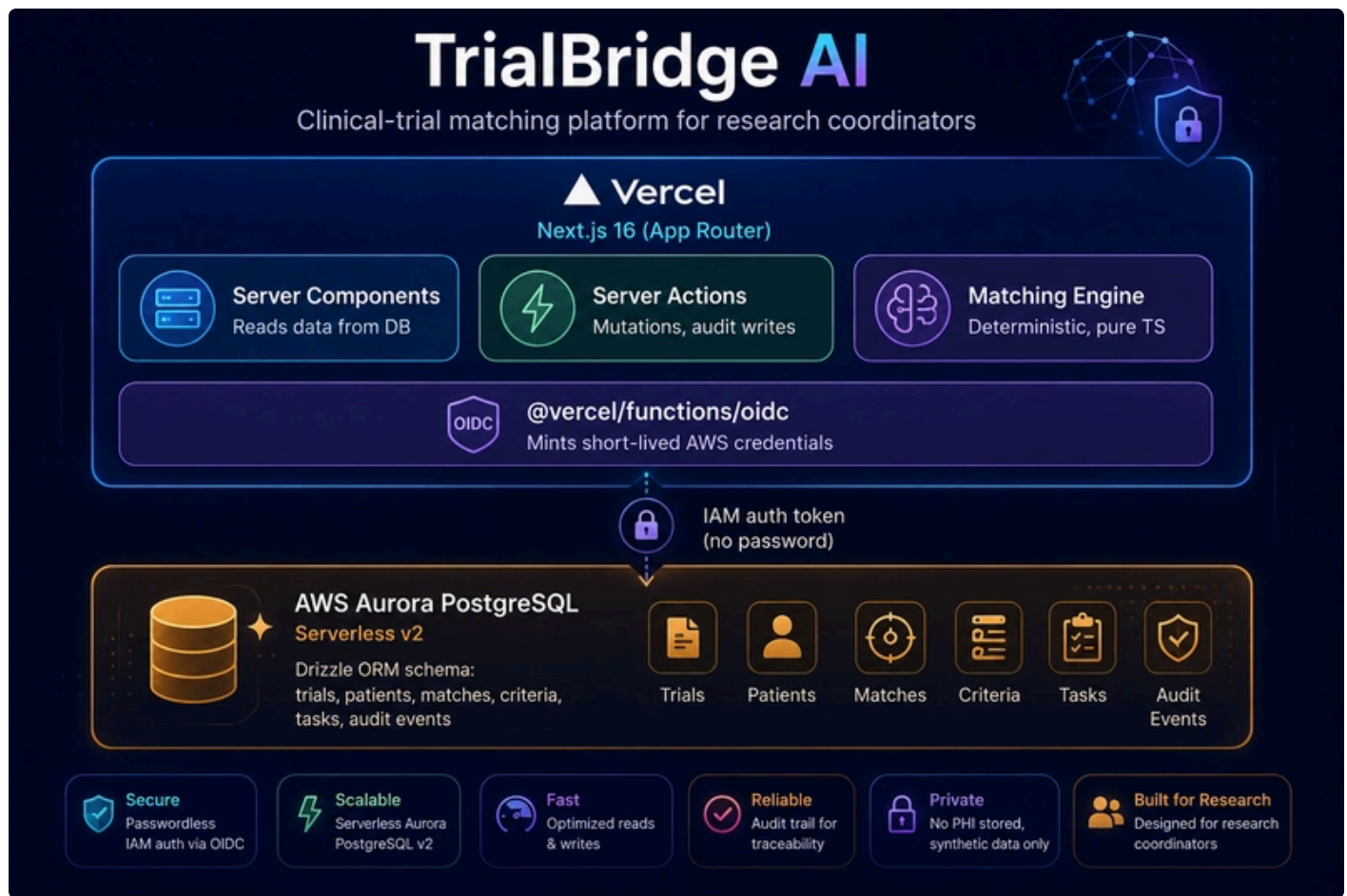
3. Why we chose Track 2: Monetizable B2B App

We deliberately built this as a **Track 2: Monetizable B2B App** rather than a consumer tool or a pure tech demo, for a few reasons:

- **A clear paying customer.** The buyers are research sites, sponsors, and CROs (contract research organizations) who already spend real budget on recruitment and coordination. Saving coordinator hours and improving enrollment rates maps directly to ROI.
- **Per-seat / per-site pricing fits the workflow.** The product is used by coordinators and PIs (principal investigators) inside an organization, so an organization-scoped, multi-user model is natural — which is exactly how the data model is shaped (organizations → users).
- **Defensibility through workflow, not magic.** The value isn't a black-box model; it's the structured, auditable pipeline from criteria to prioritized human task. That's something a business can adopt, trust, and standardize on.
- **Compliance is a feature, not an afterthought.** B2B healthcare buyers care about auditability, data residency, and "the human made the call." Building those in from day one is what makes it sellable.

4. Architecture overview

The stack is intentionally boring in the best way — managed building blocks, with the interesting logic kept in plain, testable TypeScript.



Key choices:

- **Next.js 16 App Router** with React Server Components for data-heavy pages, so reads happen on the server close to the database.
- **Drizzle ORM** for a fully typed schema and queries.
- **A deterministic matching engine** written as pure functions — no model calls in the eligibility path — so results are explainable and reproducible.
- **AWS Aurora PostgreSQL Serverless v2** as the system of record, reached with IAM authentication instead of a static connection string.

5. How we used Vercel

Vercel hosts the Next.js app and provides the runtime glue that makes passwordless database auth possible.

- **Server Components for reads.** Pages like the dashboard, trial detail, and patient detail are async server components that query Drizzle directly. There's no client-side data-fetching layer to secure or cache-bust.

- **Server Actions for writes.** Updating a coordinator task's status or writing an audit event is a server action, co-located with the UI that triggers it.
- **OIDC → AWS credentials.** This is the important one. Vercel issues a signed **OIDC token** to the deployment, and `@vercel/functions/oidc` exchanges it for short-lived AWS credentials via `sts:AssumeRoleWithWebIdentity`. No long-lived AWS access keys live in environment variables.

That last point is what connects the two halves of the stack, so it gets its own section.

6. How we used AWS Aurora PostgreSQL

Aurora PostgreSQL is the database, but the interesting part is **how we authenticate to it**. Instead of storing a database password, we use **AWS IAM database authentication**: the application asks the AWS SDK's RDS `Signer` to generate a short-lived auth token, and that token is used as the Postgres password.

Here's the actual connection setup (trimmed):

```
import { drizzle } from "drizzle-orm/node-postgres";
import { Pool } from "pg";
import { Signer } from "@aws-sdk/rds-signer";
import { awsCredentialsProvider } from "@vercel/functions/oidc";
import * as schema from "./schema";

const DB_CONFIG = {
  roleArn: process.env.DB_AWS_ROLE_ARN!, // IAM role to assume
  region: process.env.DB_AWS_REGION!,
  host: process.env.DB_PGHOST!, // Aurora cluster endpoint
  user: process.env.DB_PGUSER!, // an rds_iam-enabled DB user
  database: process.env.DB_PGDATABASE!,
  port: 5432,
};

function createPool() {
  const { roleArn, region, host, user, database, port } = DB_CONFIG;

  const signer = new Signer({
    // Vercel OIDC token → temporary AWS credentials
    credentials: awsCredentialsProvider({
      roleArn,
      clientConfig: { region },
    }),
  });
```

```

    region,
    hostname: host,
    username: user,
    port,
  });

  return new Pool({
    host,
    database,
    port,
    user,
    // The "password" is a freshly-minted IAM auth token on each connection
    password: () => signer.getAuthToken(),
    ssl: { rejectUnauthorized: false },
    max: 10,
  });
}

export const db = drizzle(createPool(), { schema });

```

The chain of trust looks like this:

1. Vercel mints an **OIDC token** for the deployment (scoped to the team, project, and environment).
2. `awsCredentialsProvider` calls **AWS STS** with that token (`AssumeRoleWithWebIdentity`) and gets temporary credentials for an IAM role.
3. The `RDS Signer` uses those credentials to generate a **time-limited Postgres auth token**.
4. `pg` connects to Aurora using that token as the password; Aurora validates it because the DB user was granted `rds_iam`.

To make this work on the AWS side, three things have to line up:

- An **IAM OIDC identity provider** for the Vercel issuer (`oidc.vercel.com/<team>`), with the correct audience.
- An **IAM role** whose trust policy allows that provider's `sub` / `aud` to assume it, plus an `rds-db:connect` permission scoped to the cluster resource ID and DB user.
- A **database user** created with `GRANT rds_iam`, and `--enable-iam-database-authentication` on the cluster.


```
-- inside the Aurora database
CREATE USER app_user WITH LOGIN;
GRANT rds_iam TO app_user;
GRANT ALL PRIVILEGES ON DATABASE trialbridge TO app_user;
GRANT ALL ON SCHEMA public TO app_user;
```

The payoff: **there is no database password anywhere** — not in env vars, not in a secrets manager, not in code. Credentials are short-lived and minted on demand.

7. Database schema

The schema is defined in Drizzle and models the full workflow from organizations down to individual audit events. The core tables:

Table	Purpose
organizations	The B2B tenant (research site, sponsor, CRO).
users	Coordinators / PIs, scoped to an organization.
trials	Clinical trials: condition, phase, location, sponsor, status, summary.
trial_criteria	Structured inclusion/exclusion rules (field , operator , value , weight).
synthetic_patients	Synthetic patient profiles (demographics, condition, attributes).
patient_labs	Lab results linked to a patient.
trial_matches	A patient↔trial evaluation with status, score, and summary.
match_explanations	Per-criterion result for a match (met , missing , failed , triggered_exclusion).
coordinator_tasks	Human follow-up work with status + priority.
audit_events	Append-only record of who did what.

A representative slice, in Drizzle:

```
export const trialCriteria = pgTable("trial_criteria", {
  id: uuid("id").primaryKey().defaultRandom(),
  trialId: uuid("trial_id")
    .notNull()
    .references(() => trials.id, { onDelete: "cascade" }),
  type: criterionTypeEnum("type").notNull(), // 'inclusion' | 'exclusion'
  field: text("field").notNull(),           // e.g. "age", "hba1c"
  operator: text("operator").notNull(),      // e.g. "gte", "in", "contains"
  value: text("value").notNull(),            // e.g. "18", "male|female"
  description: text("description").notNull(),
  weight: integer("weight").notNull().default(1),
});
```

Two design notes worth calling out:

- **Criteria are data, not code.** Each rule is a row with a `field`, `operator`, `value`, and `weight`. That means trials can be added or edited without touching the matching logic.
- **Explanations are first-class.** `match_explanations` stores *why* each criterion passed or failed, so the UI can show the reasoning and the result stays auditable.

8. Trial matching workflow

The matching engine (`src/lib/matching/engine.ts`) is **pure, deterministic TypeScript** — no LLM in the eligibility decision. It takes a patient profile and a trial's criteria and returns a status, a score, and per-criterion evaluations.

Each criterion is evaluated independently:

```
function evaluateCriterion(patient, criterion): CriterionEvaluation {
  const rawValue = getFieldValue(patient, criterion.field);

  // No data? That's "missing" — a flag for the coordinator, not a rejection.
  if (rawValue === undefined || rawValue === null || rawValue === "") {
    return { criterionId: criterion.id, result: "missing", explanation: "..." };
  }

  const passes = compareValues(rawValue, criterion.operator, criterion.value);

  if (criterion.type === "exclusion") {
    return passes
      ? { result: "triggered_exclusion", /* ... */ } // hard stop
  }
}
```



```

      : { result: "met", /* ... */ };
    }

    return passes
      ? { result: "met", /* ... */ }
      : { result: "failed", /* ... */ };
  }

```

The overall **status** is then derived conservatively:

- Any **triggered exclusion** → `not_eligible`.
- Any **failed inclusion** → `not_eligible`.
- All inclusions **met** → `likely_eligible`.
- Some inclusions **missing** (but none failed) → `possibly_eligible` — i.e. "a human needs to fill in the gaps."

The **score** is a weighted percentage of inclusion criteria met, where missing data counts as half credit, and a triggered exclusion caps the score low:

```

if (evaluation.result === "met")      earned += criterion.weight;
else if (evaluation.result === "missing") earned += criterion.weight * 0.5;
// ...
return Math.round((earned / totalWeight) * 100);

```

This design has a deliberate bias: **missing information never silently disqualifies a patient**. It surfaces as `possibly_eligible` with an explicit "coordinator should verify" note, because the worst outcome in recruitment is quietly dropping someone who actually qualified.

9. Safety and privacy design

Because this touches healthcare, safety wasn't a feature we added at the end — it constrained the whole build.

- **Synthetic data only.** Every patient in the system is fabricated. There is no real PHI, by design, in the demo.
- **No diagnosis, ever.** The product surfaces structured assessments and explanations; it never outputs a medical decision. A persistent disclaimer component makes this explicit in the UI:

Research workflow demonstration only. Not for diagnosis or treatment decisions. All patient data is synthetic. TrialBridge AI supports research coordinators in organizing information and preparing cases for human review — it does not diagnose, treat, prescribe, or make final medical decisions.

- **Human in the loop by construction.** Matches become *tasks*, not actions. The terminal state of a positive match is `referred_to_pi` — a human, not the system.
- **Deterministic, explainable logic.** No black-box model decides eligibility. Every status has a traceable per-criterion explanation.
- **Auditability.** `audit_events` records actions for after-the-fact review — exactly what a B2B compliance reviewer expects.
- **Passwordless data access.** As covered above, there's no database password to leak; credentials are short-lived IAM tokens minted via OIDC.

10. What we learned

A few takeaways from building TrialBridge AI:

1. **OIDC-based database auth is worth the setup cost.** Wiring up the IAM identity provider, role trust policy, `rds-db:connect` permission, and `rds_iam` database user took the most iteration of anything in the project. The reward — zero stored database credentials — is absolutely worth it for a B2B product where security posture is part of the sale. The most common failure mode was a **trust-policy mismatch** (e.g. the role allowing `production` but the deployment running as `development`); matching the OIDC `sub/aud` exactly is the whole game.
2. **Region and account consistency matters.** The IAM role and OIDC provider are global, but the RDS cluster, the `AWS_REGION` env var, and the `rds-db:connect` resource ARN must all reference the same region and account. A mismatched default CLI region sent us looking for a "missing" cluster that was simply in another region.
3. **Keep the high-stakes logic deterministic.** For an eligibility decision, plain, testable functions beat a model call. They're explainable, reproducible, and easy to audit — and in a regulated domain those properties are the product.
4. **Model "missing" as its own state.** The difference between "doesn't qualify" and "we don't have the data yet" is the difference between losing a patient and flagging a quick coordinator follow-up. Encoding `missing` as a first-class result changed the whole tone of the output.

5. Design for the human handoff first. The most useful artifact isn't the match — it's the prioritized, explained task that lands on a coordinator's desk. Building the workflow (tasks, statuses, audit) around that handoff is what makes it a product instead of a demo.

TrialBridge AI is built with Next.js 16, Drizzle ORM, and AWS Aurora PostgreSQL, deployed on Vercel. All patient data is synthetic, and the application is a research-workflow tool — not a medical device.

Note: This article was created for the purpose of entering the H0: Hack the Zero Stack with Vercel v0 and AWS Databases hackathon.

Written by Vandana Yarala & Kamalakara Thota. Questions or feedback? Contact us at [<https://www.linkedin.com/in/vandanayarala/>].

Top comments (0)

[Code of Conduct](#) • [Report abuse](#)

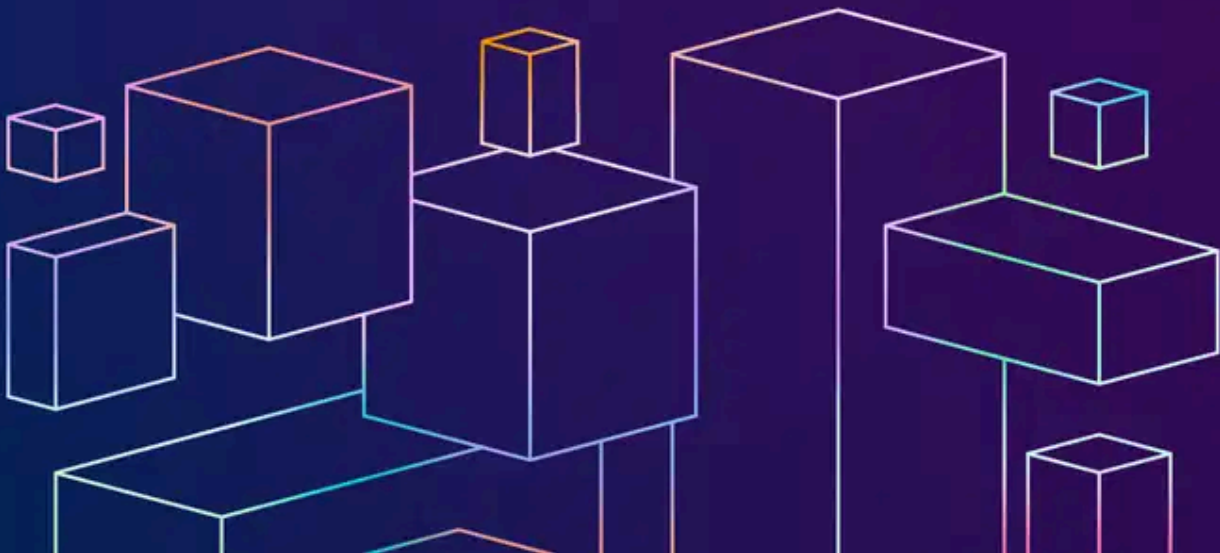


AWS PROMOTED



Watch Industries LIVE!

Building industry breakthroughs together



Building industry breakthroughs together

Discover how the cloud helps businesses adapt, innovate, and grow in real time. Tune in live.

[Register Now](#)



Vandana Yarala

Senior Architect

LOCATION

Houston, Texas

JOINED

27 Jun 2026

Trending on DEV Community 🔥



Thank you DEV community: the Thinking Engineer Toolkit is live

#ai #softwareengineering #developer #productivity



What's Wrong with DEV.to Notifications?

#discuss #coding #developer



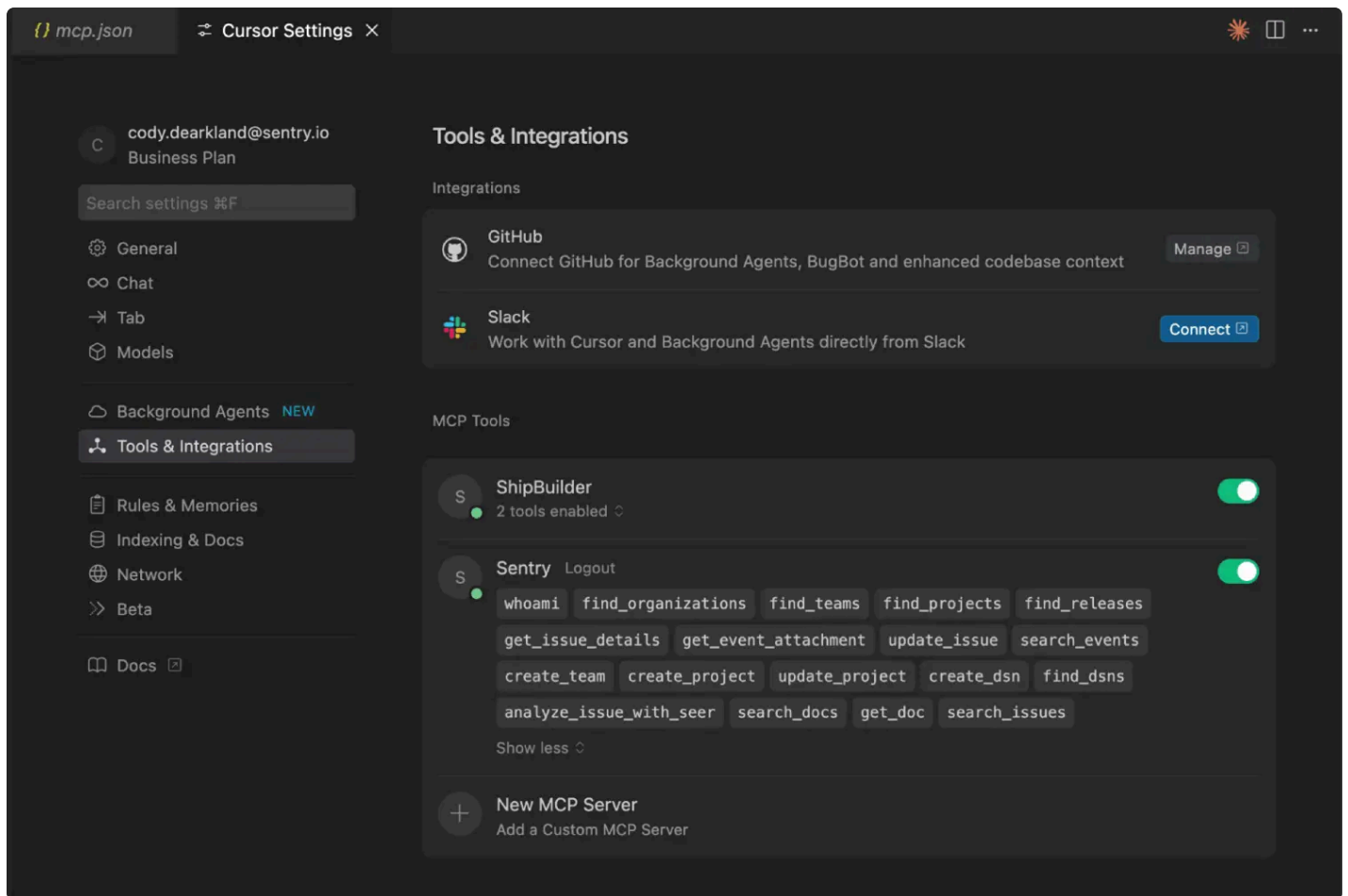
How I Used Automated Red Teaming to Evaluate My AI Agent's Safety

#agents #ai #security #aws



Sentry PROMOTED





Smarter debugging with Sentry MCP and Cursor

No more copying and pasting error messages, logs, or trying to describe your distributed tracing setup or stack traces in chat. MCP can investigate real issues, understand their impact, and suggest fixes based on the actual production context.

[Read more →](#)